

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

“ДЗ 2: Проектирование и технический дизайн API”

Выполнил:

Захматов Юрий

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
2. Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
3. Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4. Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

1. Функциональные требования

1.1. Модуль аутентификации и управления пользователями

Система должна обеспечивать регистрацию новых пользователей с использованием email, username и пароля. Пароли должны храниться только в хешированном виде с использованием алгоритма BCrypt. Аутентификация пользователей должна осуществляться по username и паролю с выдачей JWT токена доступа, который имеет время жизни 24 часа. Пользователь должен иметь возможность обновлять свой профиль, включая имя, фамилию, отчество и номер телефона, а также изменять пароль. В системе должны различаться две роли пользователей: арендатор (renter) и арендодатель (landlord). Один пользователь может выступать в обеих ролях одновременно.

1.2. Модуль управления недвижимостью

Арендодатель должен иметь возможность создавать объявления о сдаче недвижимости, указывая заголовок, описание, цену за сутки, площадь, тип недвижимости (квартира, дом, комната и т.д.), полный адрес с указанием страны, региона, города, улицы, номера дома, почтового индекса, а также координаты для отображения на карте. К объявлению можно загружать несколько фотографий. Арендодатель имеет право редактировать и удалять только свои объявления. Все пользователи могут просматривать список

объявлений и детальную информацию по каждому объявлению, включая фотографии, описание, условия аренды и средний рейтинг.

1.3. Модуль поиска и фильтрации

Система должна предоставлять возможность поиска объявлений с фильтрацией по типу недвижимости, ценовому диапазону, городу или району, площади, а также поиск по заголовку или описанию. Результаты поиска должны поддерживать сортировку по цене (возрастание/убывание), дате добавления и площади. Все списковые запросы должны использовать пагинацию с настраиваемым размером страницы.

1.4. Модуль бронирования

Арендатор должен иметь возможность создавать запросы на бронирование объекта на выбранные даты. Система должна автоматически проверять доступность указанных дат и рассчитывать общую стоимость как произведение цены за сутки на количество дней. Арендодатель может подтвердить или отклонить поступивший запрос на бронирование. Арендатор может отменить свой запрос до момента его подтверждения. После подтверждения бронирования арендатор должен иметь возможность произвести оплату.

1.5. Модуль платежей

Система должна отслеживать статус платежей по каждому бронированию, поддерживая статусы PENDING, COMPLETED, FAILED, REFUNDED. Пользователь должен иметь возможность просматривать историю всех своих платежей с детализацией по каждому бронированию.

1.6. Модуль отзывов и рейтингов

После завершения бронирования арендатор может оставить отзыв на объект, указав оценку от 1 до 5 и текстовый комментарий. Отзыв можно редактировать или удалять. На странице объекта должна отображаться средняя оценка, рассчитанная на основе всех отзывов.

1.7. Модуль сообщений

Пользователи должны иметь возможность обмениваться сообщениями по конкретному объекту недвижимости. Все сообщения сохраняются с

временной меткой. Пользователь видит список своих чатов с возможностью перехода к переписке. В чате должна быть реализована пагинация для подгрузки истории сообщений.

1.8. Модуль личного кабинета

В личном кабинете пользователь должен видеть список своих объявлений (если выступает в роли арендодателя), список своих бронирований с указанием статусов (если выступает в роли арендатора), историю всех своих платежей, а также активные и завершенные чаты.

2. Нефункциональные требования

2.1. Безопасность

Все эндпоинты API, кроме эндпоинтов регистрации и входа, должны быть защищены JWT токеном, который передается в заголовке Authorization. Пользователь может изменять или удалять только ресурсы, принадлежащие ему. Система должна валидировать все входные данные и возвращать понятные сообщения об ошибках.

2.2. Надежность и доступность

Система должна корректно обрабатывать ошибки на всех уровнях и возвращать клиенту понятные сообщения с соответствующими HTTP статус кодами.

2.3. Масштабируемость

API должен быть stateless, что позволит горизонтально масштабировать приложение путем добавления новых экземпляров. Вся информация о состоянии пользователя должна храниться в JWT токене.

2.4. Документация

Весь API должен быть задокументирован в формате OpenAPI 3.0. Документация должна быть доступна через Swagger UI по пути /swagger-ui.html. Каждый эндпоинт должен содержать описание, примеры запросов и ответов, а также описание возможных ошибок.

2.4. Технологический стек

Приложение должно быть реализовано на Java с использованием Spring Boot. Для работы с базой данных используется Spring Data JPA и Hibernate. База данных — PostgreSQL. Аутентификация реализована с использованием Spring Security и JWT. Сборка проекта — Gradle.

3.

В качестве формата реализации API был выбран REST с использованием HTTP.

Вывод

В ходе выполнения домашней работы были описаны функциональные и нефункциональные требования к проекту. Также были разработаны эндпоинты в формате OpenAPI-схемы.